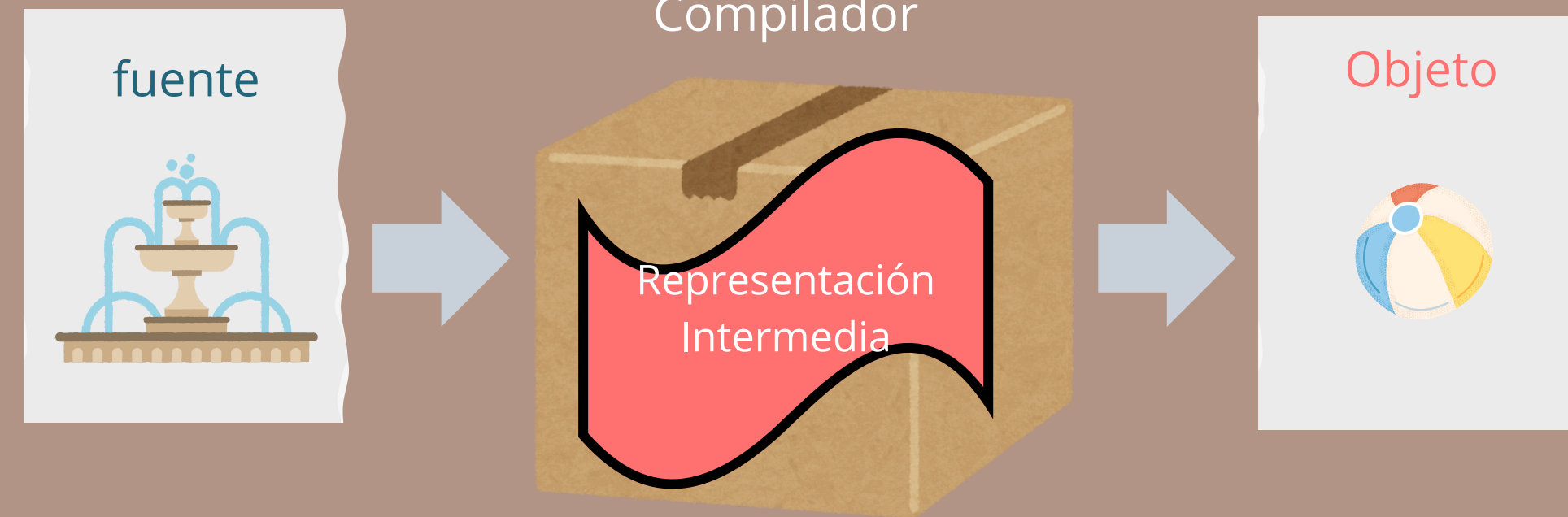
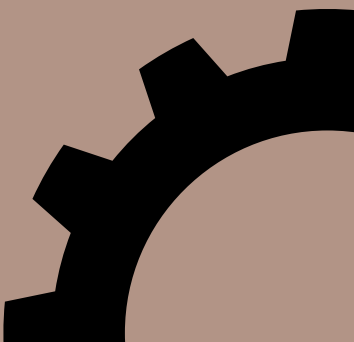


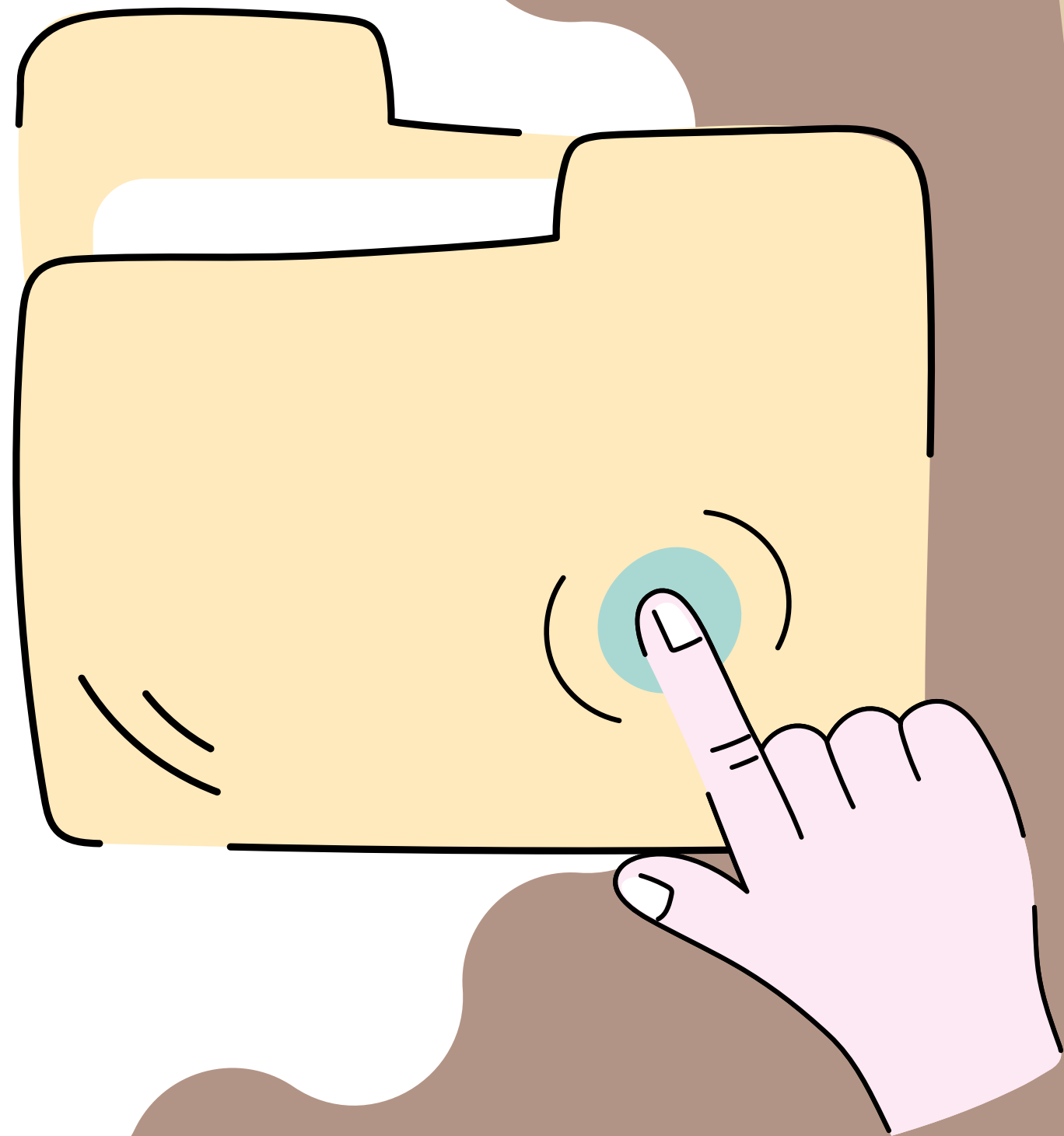


Apuntes Compiladores e Intérpretes

Clase 3 de setiembre
2026 (03/09/2026)

Ian Sebastian Ugalde Muñoz -2024124076





Contenidos

- Compilador y qué es la representación intermedia
- Analizar y sintetizar
- Fases de compilación: Datos generales
- Fases de compilación al detalle:
 - Scanner (Front End)
 - Parser (Front End)
 - Analizador Semántico (Front End)
 - Optimizador de fuente. (Front End)
 - Generador de Código (Back End)
 - Optimizador de Código (Back End)

Representación Intermedia

Es el lenguaje intermedio en el cual se comunican o se conectan el Backend y el Frontend.

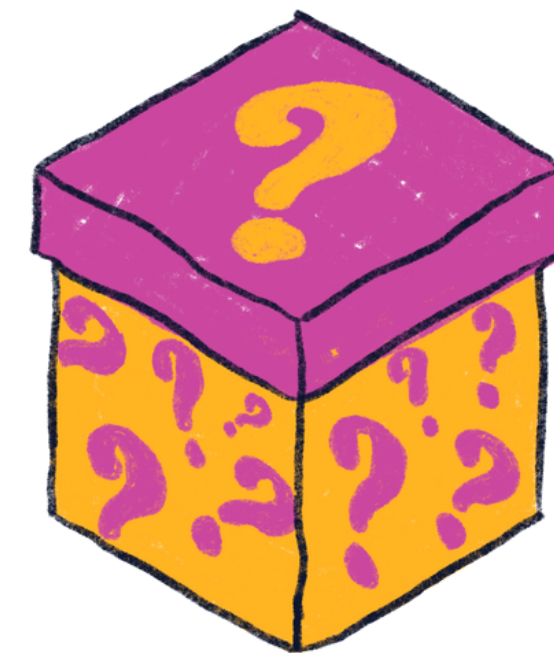
El proceso de compilación se puede definir a grandes rasgos en dos fases:

- **Frontend:** Agarra el fuente y lo lleva a una representación intermedia
- **Backend:** Le entra la representación intermedia y sale el Objeto

Compilador

Un chunche que recibe un lenguaje (fuente) y lo traduce a otro lenguaje diferente (objeto). Obviamente hay un montóoon de cosas intermedias

cosas
intermedias



Analizar Y Sintetizar

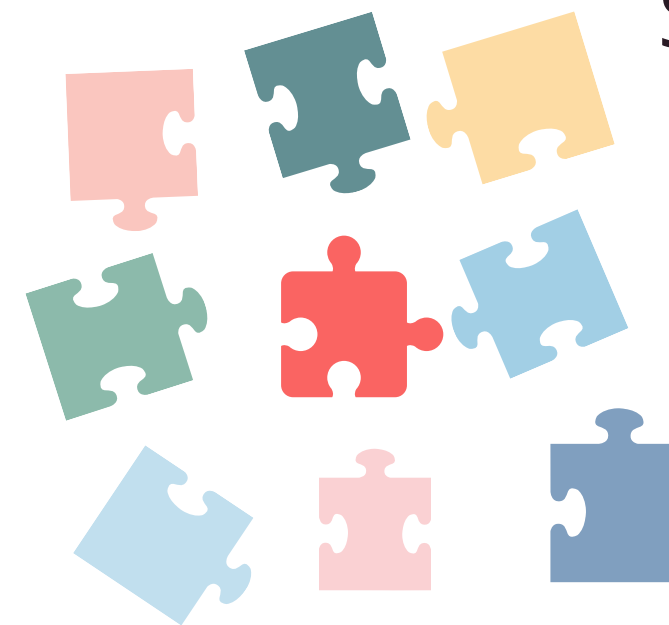
Analizar: Tomar ALGO Y
DESCOMPONERLO EN PARTES para
entenderlo

Sintetizar: construir algo a partir de
varias piezas sueltas

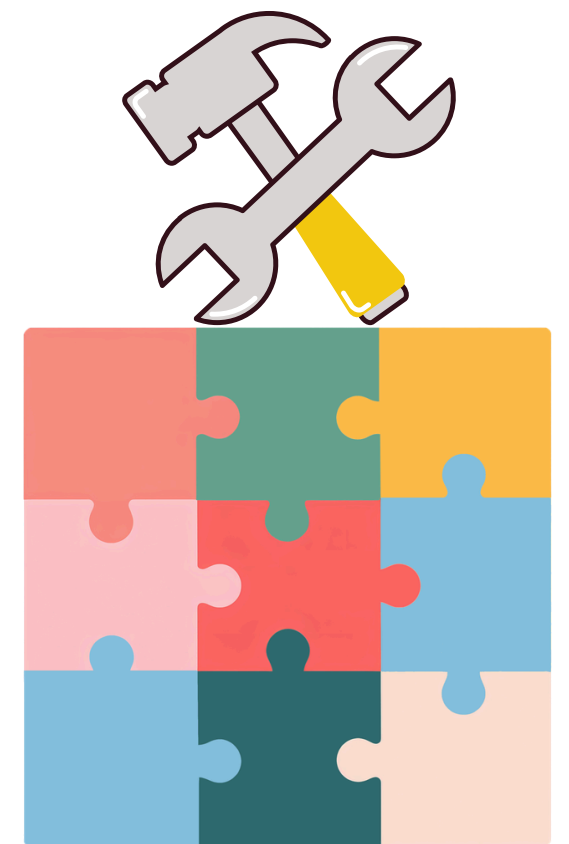
ejemplo: un rompecabezas, analizarlo seria desarmarlo
y analizar cada piecita para ver un pedacito en
especifico y entenderlo. Sintetizarlo es agarrar el
montón de piecitas y empezar a armar el lienzo o la
figura.



Analizar



Sintetizar



Y cómo relacionamos el análisis y síntesis con compiladores?

Fácil, estas operaciones se notan en el compilador.

Dónde?

El frontend se encarga de analizar.

Y el Backend es el que sintetiza.

¡¡Algo importante!!

Entre más independientes sean estos dos más flexibilizada va a haber. Aunque ojo en la práctica al final uno termina dejando aunque sea un poquito dependiente a uno del otro

Y hay que recordar que lo único que tienen en común el backend y el frontend es la representación intermedia.

DON'T FORGET

Qué hace ACK?

ACK tiene un montón de Frontends, pero solo
tiene un Backend

Posee una tabla para cada arquitectura
Y tiene un machote para cada uno para escupir

COMPUTING
PRACTICES

A PRACTICAL TOOL KIT FOR MAKING PORTABLE COMPILERS

ANDREW S. TANENBAUM, HANS VAN STAVEREN, E. G. KEIZER, and
JOHAN W. STEVENSON Vrije Universiteit, Amsterdam, The Netherlands

Ejemplos de machotes en otros contextos:

Machote pargo



Machote
documento



“El macho”



Fases de Compilación:

Cositas importantes

Pasada: Una lectura completa del fuente de inicio a fin
"N pasadas" osea leer N veces el fuente de inicio a fin

Algunas definiciones pueden estar fisicamente a su uso.
Tener más pasadas significa que hay más flexibilidad
Peroooo....

Más pasadas también significa más tiempo de compilación



¡¡Recordemos!!

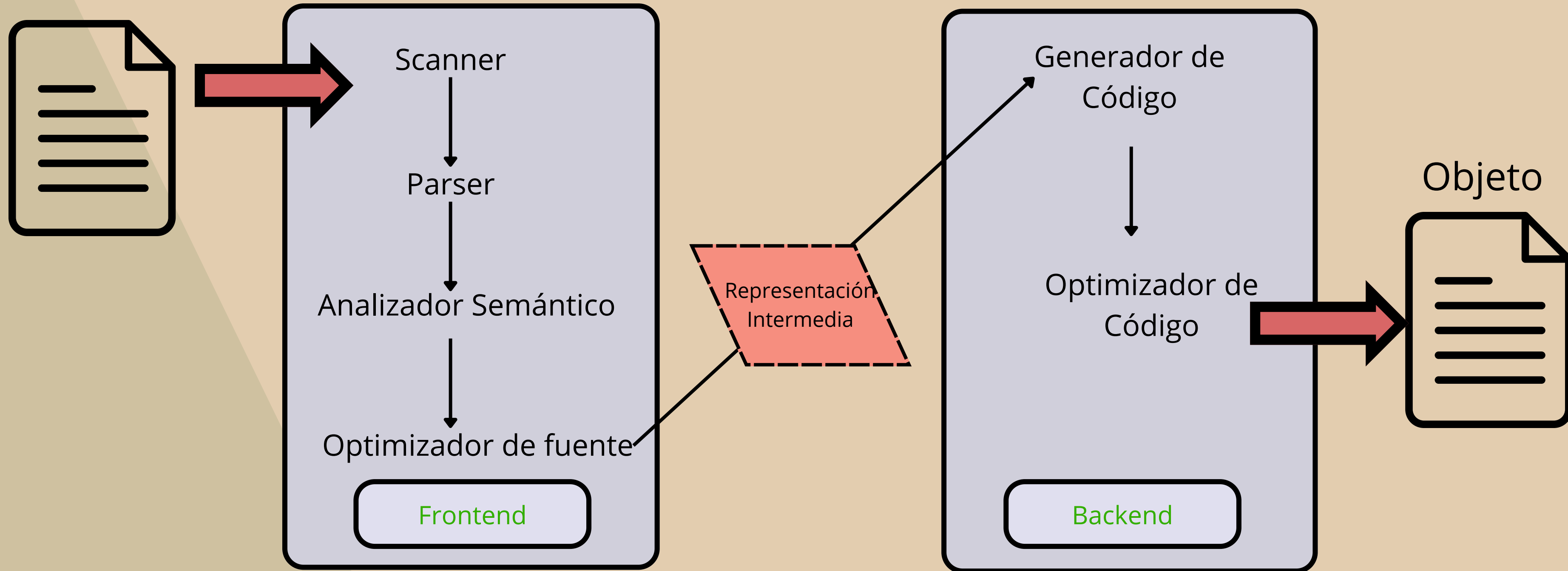
Estructura básica: Backend y Frontend

Fases de Compilación:

¡Aquí inicia la mentira!



Fuente



Fases de Compilación:

Scanner (Front End)

Parser (Front End)

Analizador Semántico (Front End)

Optimizador de fuente. (Front End)

Generador de Código (Back End)

Optimizador de Código (Back End)

**Detallemos en cada
uno de estos.**



Frontend del Compilador

Scanner

Análisis Léxico

El analizador léxico es el primero que ve el fuente.

Léxico ? = Vocabulario

Vamos a decir que toma el de entrada y lo descompone en unidades lexicas minimas ("tokens")

Este es implementado en automatas de Estados Infinitos.

La buena noticia: Ya tenemos mucha experiencia y hoy en día hay herramientas que generan la mayor parte de un scanner

Comentarios interesantes de la clases:

Los seres humanos somos muy capaces y somos los únicos que tenemos un lenguaje como tal (los monos no). Y lo primero que vamos desarrollando es nuestro "scanner".

Y sí, somos tan chuzos que tenemos lenguaje de señas.



Scanner

Análisis Léxico

```
int main() {  
    int edad = 18;  
    const float PI = 3.14159;  
    if (edad >= 18) {  
        printf("Mayor de edad\n");  
    }  
    return 0;  
}
```

Ejemplo de que veria el scanner si le encantasen los colores:

Verde: Constantes

Celeste: operadores

Amarillo: Simbolos y separadores

Rojo: Nombres ingresados por el programador

Y el **fucsia:** para palabras reservadas.

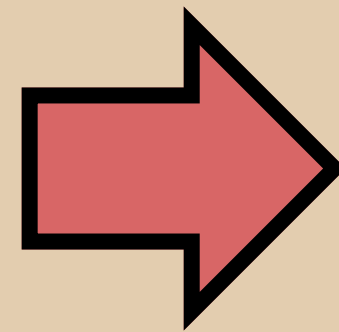
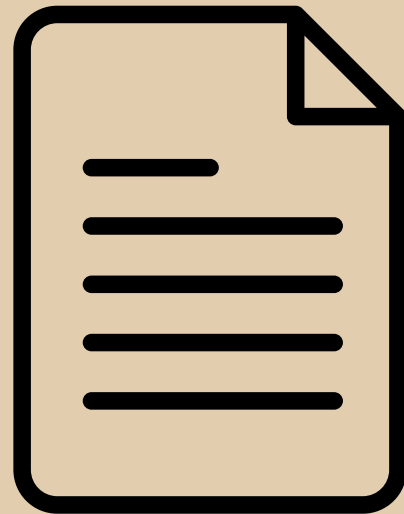


Scanner

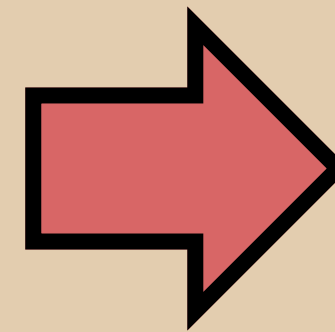
Análisis Léxico

El scanner agarra el fuente y lo despedaza para convertirlo en unidades más pequeñas o “tokens”

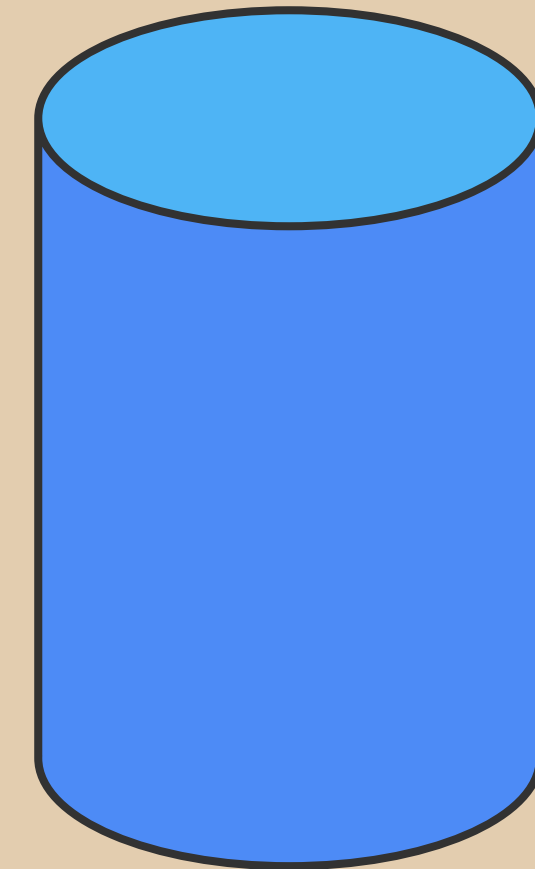
Fuente



Scanner



Tokens



Dato curioso: Cuesta que se caiga, pero para hacerlo hay que meterle un símbolo raro, algo fuera del UTF-8 o algo así que no logre reconocer.



Parser

Análisis Sintáctico

El análisis Sintáctico se encarga de revisar que la sintaxis de un programa este correcta.

Cómo lo hace? Toma la secuencia de **tokens** (que recibimos de scanner) y verifica que correspondan a construcciones válidas
Algo importante es que requiere una gramática.

Gramática: conjunto de reglas y principios que gobiernan el uso de un idioma y la forma en que se organizan las palabras para crear oraciones con sentido. En nuestro caso se trata del lenguaje de programación.

Normalmente son implementados con “Push-down Automata”

La buena noticia: También tenemos ya mucho tiempo con estos y hay herramientas hoy en día que generan la mayor parte de un parser.

Comentarios interesantes de la clases:

¿Y si viniesen extraterrestres pensarían que hablamos el mismo lenguaje?

-Es lo más seguro todos los idiomas tienen verbos, sujetos, predicados, sustantivos, etc....

-Hablan lo mismo “Lenguajes Humanos”.

Recomendación cinematográfica: The arrival



Parser

Análisis Sintáctico

El parser se encarga de armar el árbol a partir de los tokens. Cuando no logra armarlo, por eso tira error sintáctico nuestro compilador

Algoritmo de Euclides:

Ejemplo:

while $b \neq 0$

if $a > b$

$a := a - b$

else

$b := b - a$

return a



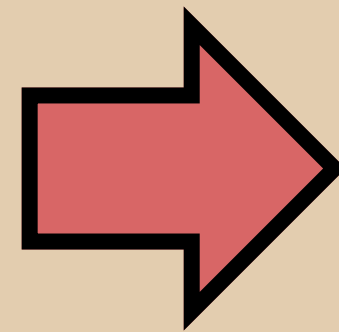
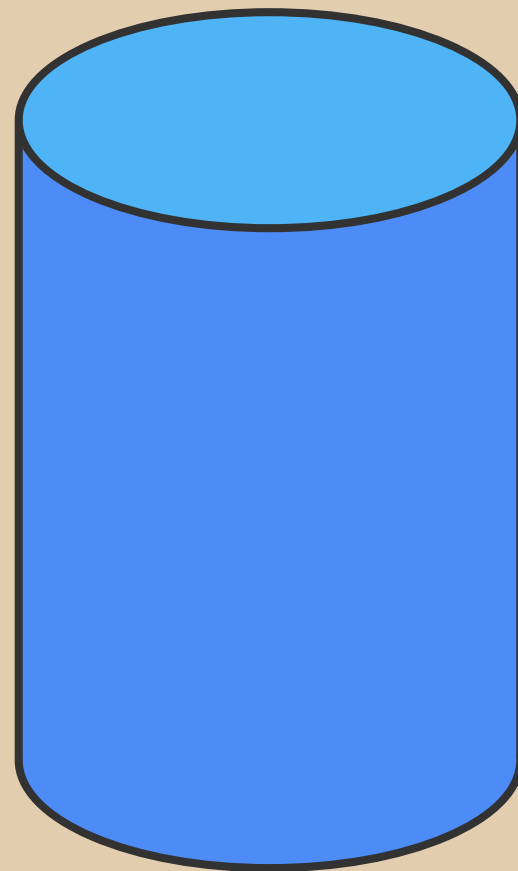
Parser

Análisis Sintáctico

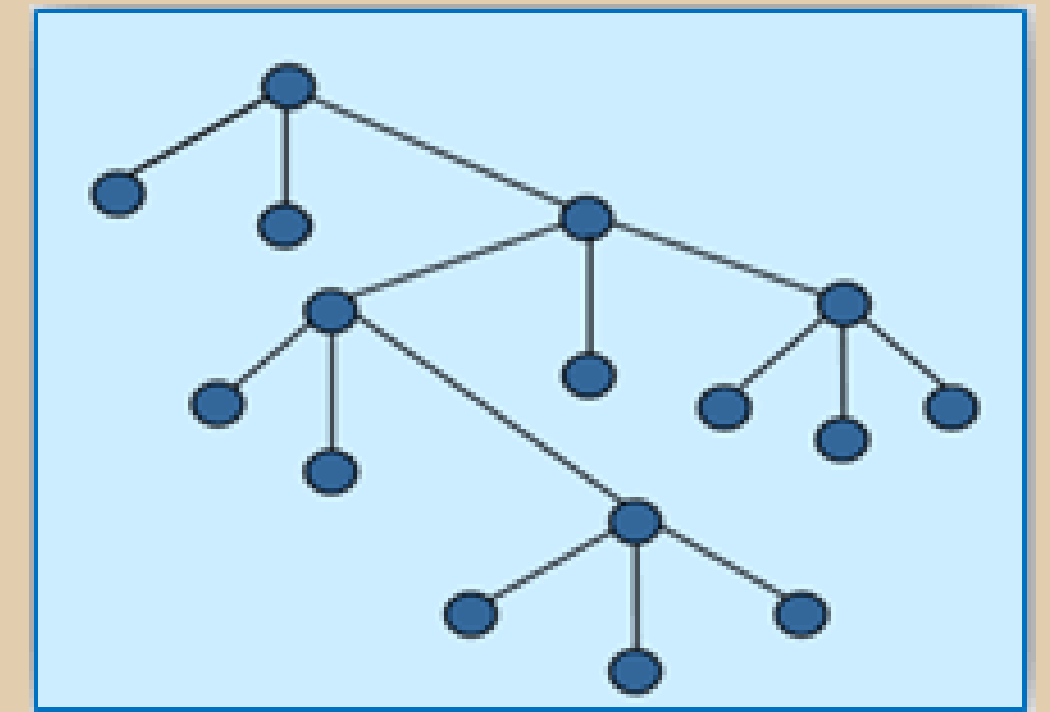
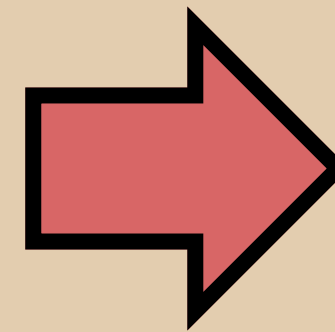
Entonces tenemos: El parser recibe un puñado de tokens y los convierte en un hermoso árbol sintáctico

Tokens

Árbol Sintáctico



Parser



Analizador Semántico



Este se encarga de ver árboles.

Aunque la sintáxis puede estar correcta, también puede que no signifique nada
El analizador semántico tiene que entender el significado del programa.



Ejemplos de sintáxis correcta, pero que aún así puede no signicar algo:
Un círculo cuadrado corría lentamente hacia el pasado mañana.
La dulce sal de tus palabras me quemó con su frío intenso.

Analizador Semántico



Se encarga de:

Revisar si tiene sentido lo que pide que se haga
Determina el comportamiento del código en tiempo de corrida.

Analiza semántica estática:

Declaraciones, revisión de tipos, etc...

Y lo más importante...

Se encarga de construir la **Tabla de Símbolos**, y a su vez la consulta y la actualiza.



Como programadores tenemos que ponernos el sombrero de compiladores:

Cada warning significa: ¡¡tenga cuidado!! estoy bateando ...¿está segur@?

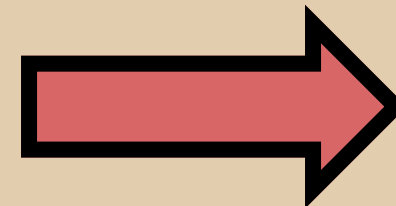
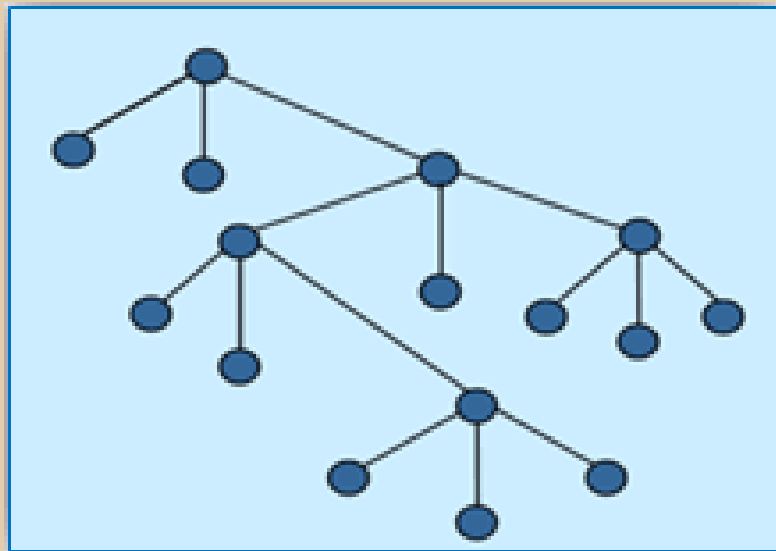
-¿Y por qué? Porque algo puede tener más de un sentido.

Debemos ser específicos y demostrarle al compilador que sabemos lo que hacemos. O talvez no sabemos, pero hay que parecer cool.

Analizador Semántico



El analizador Semántico toma el árbol de sintáctico y crea el árbol decorado o de atributos y crea también la tabla de símbolos



Analizador Semántico

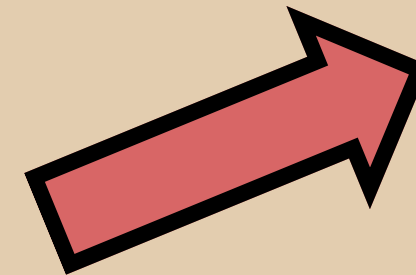
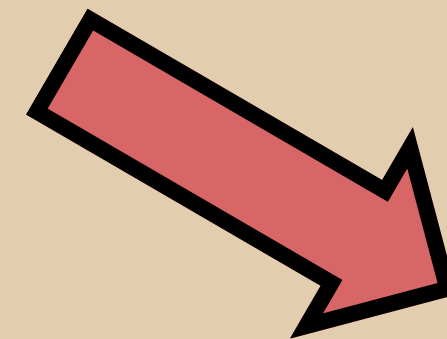
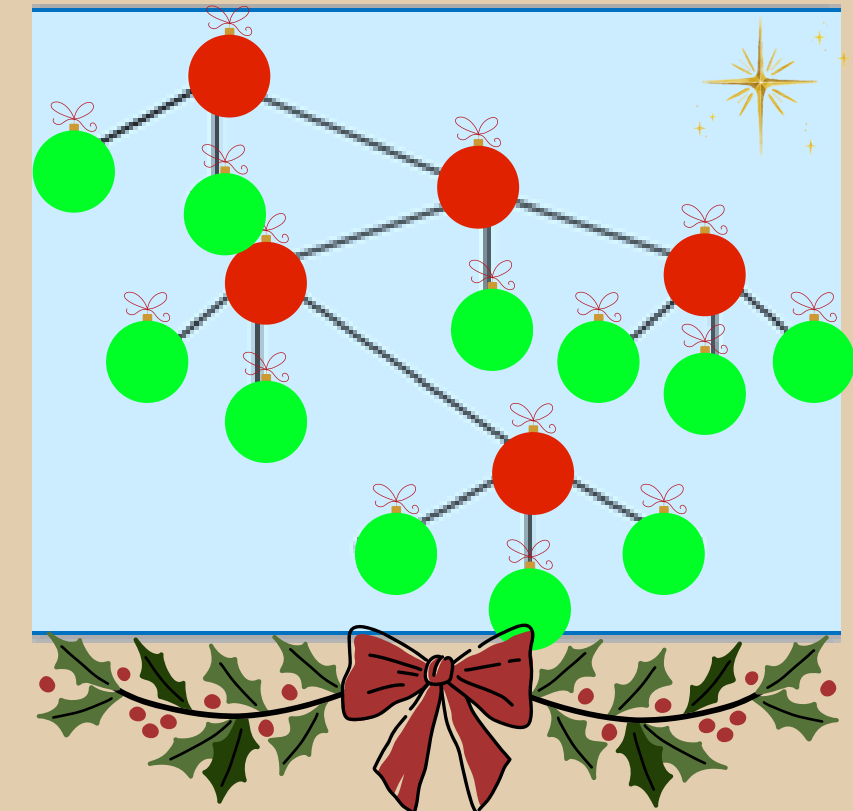


Tabla de Símbolos

DECLARACIÓN	TIPO	DIMENSION
VAR+0	C	10
VAR+10	I	0
VAR+12	B	0
VAR+13	F	3

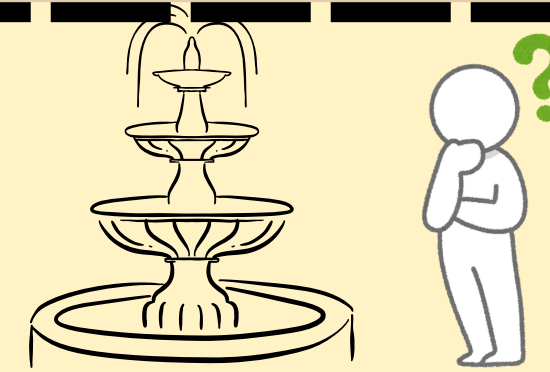


Árbol decorado



Árbol decorado = ¿¿Árbol navideño??

Optimizador de Fuente



Se encarga de mejorar el código donde se pueda.
Y es el responsable de hacer la representación intermedia.

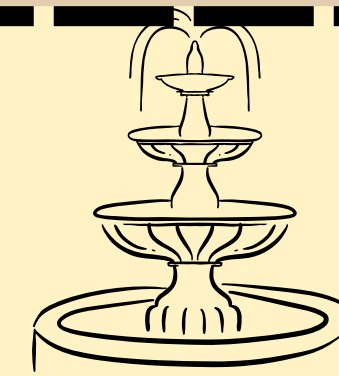


Se encarga de:

- Hacer el calculo de constantes y su propagación.
- Eliminación de código innecesario (ojo porque puede ser incluso funciones completas, cosas que nunca se van a usar).

Esto se puede hacer con un árbol igual, ya que es fácil convertir el árbol en una estructura lineal y es relativamente similar a un ensamblador de 3 direcciones.

Optimizador de Fuente



Ejemplo de
código de
representación
intermedia:

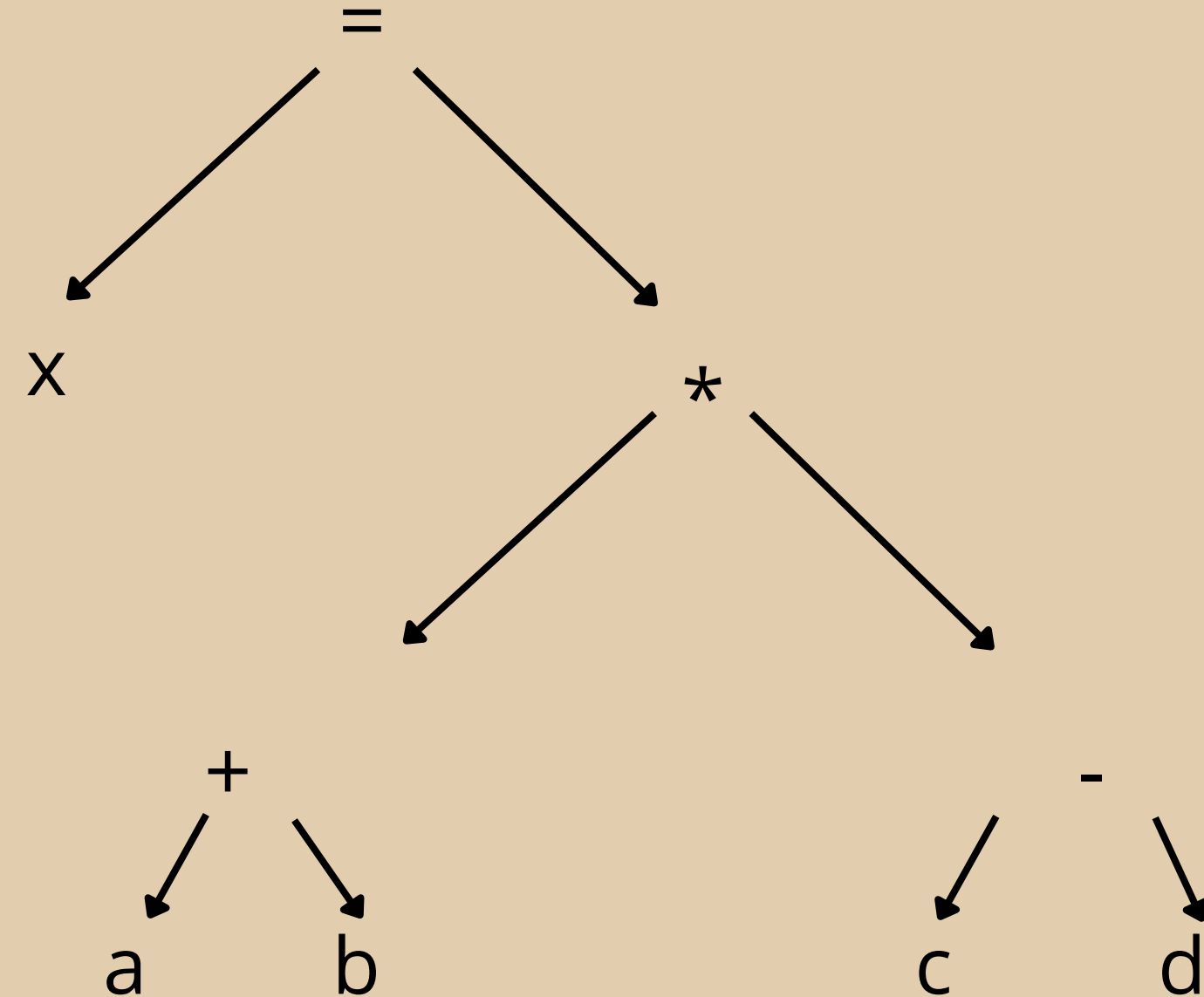
$x = (a + b) * (c - d);$

$t1 = a + b$

$t2 = c - d$

$t3 = t1 * t2$

$t4 = x = t3$

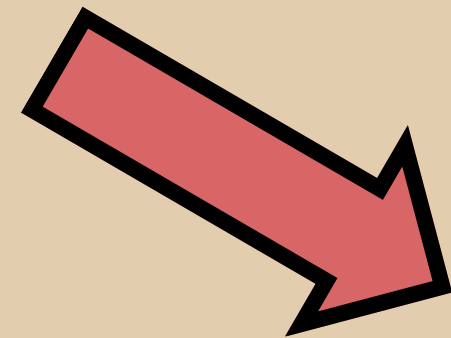
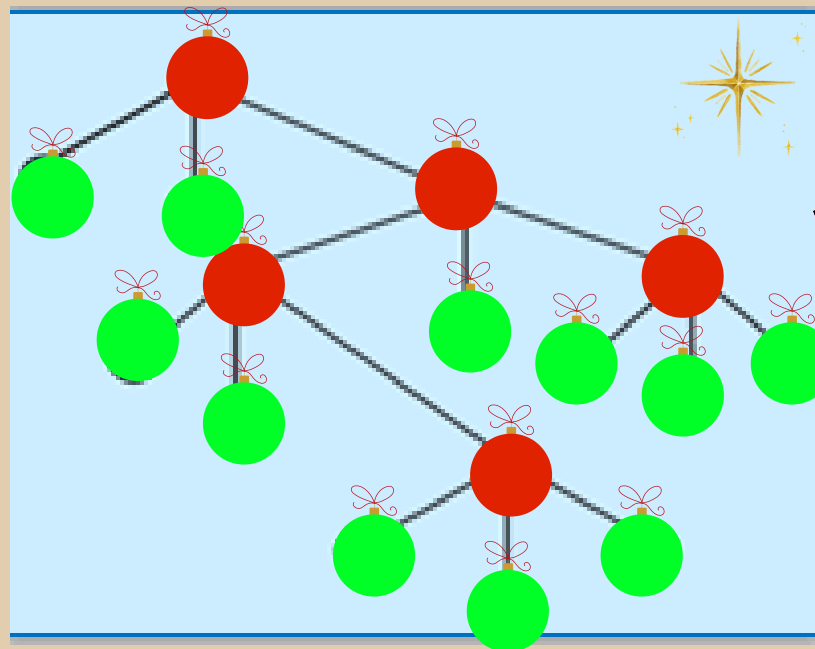


Optimizador de Fuente

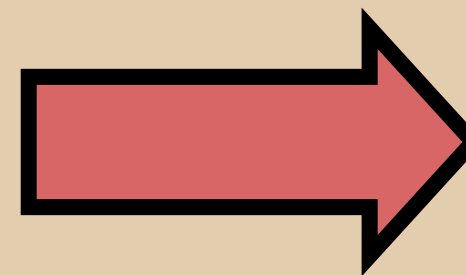
Tabla de Símbolos

DECLARACIÓN	TIPO	DIMENSION
VAR+0	C	10
VAR+10	I	0
VAR+12	B	0
VAR+13	F	3

Árbol decorado



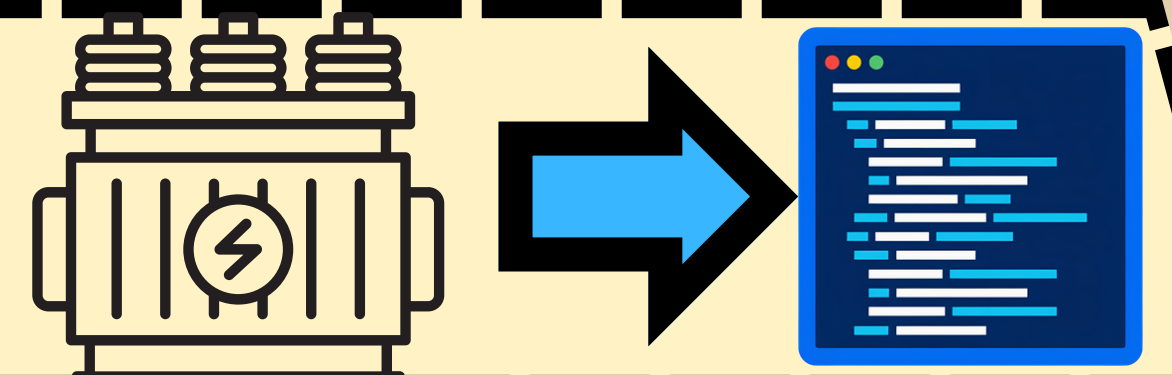
Generación IR



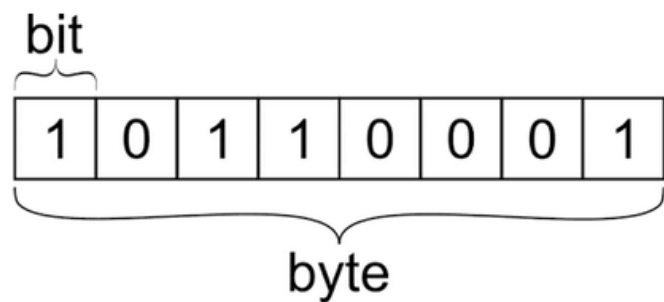
Representación
Intermedia

Backend del Compilador

Generador de Código



Se encarga de agarrar el código intermedio y convertirlo en código real.



Representación de datos muy importante.
Saber la cantidad de bytes
Y las restricciones de colocación

Hay que realmente conocer muy bien y explotar todas las posibilidades que nos brinda la arquitectura

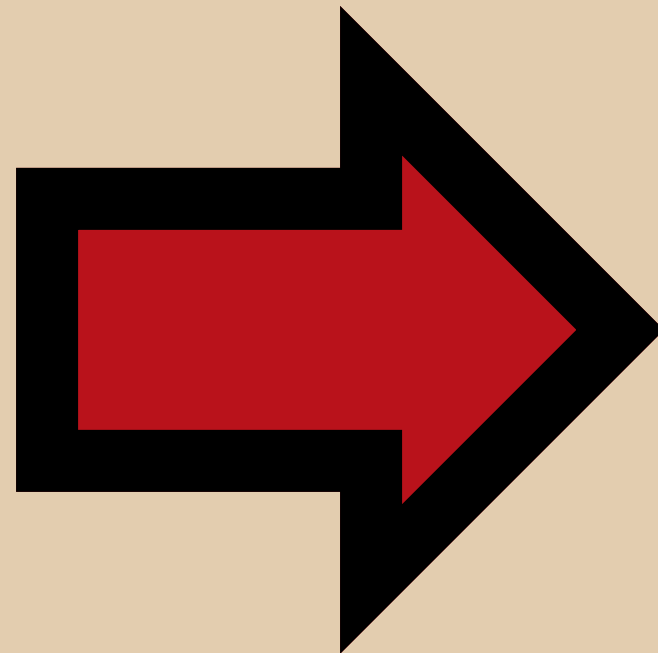


nota: nose si aplica en compiladores.

Generador de Código

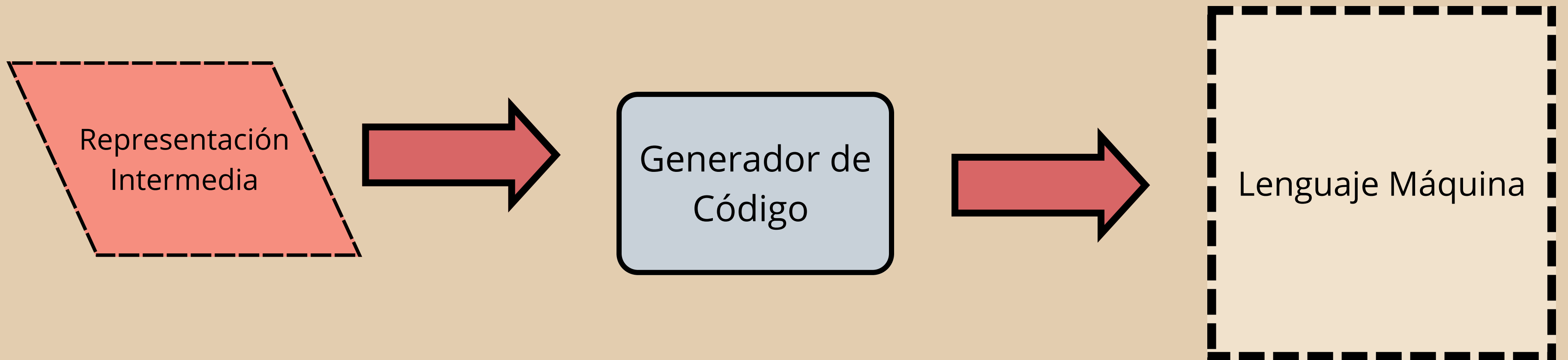
A pasar nuestro código intermedio a x86:

```
■ x = (a + b) * (c - d);  
■  
■ t1 = a + b  
■ t2 = c - d  
■ t3 = t1 * t2  
■ t4 = x = t3
```



```
■ ; t1 = a + b  
■ MOV AX, [a]  
■ ADD AX, [b]  
■ MOV [t1], AX  
■  
■ ; t2 = c - d  
■ MOV AX, [c]  
■ SUB AX, [d]  
■ MOV [t2], AX  
■  
■ ; t3 = t1 * t2  
■ MOV AX, [t1]  
■ MUL [t2]  
■ MOV [t3], AX  
■  
■ ; x = t3  
■ MOV AX, [t3]  
■ MOV [x], AX
```

Generador de Código

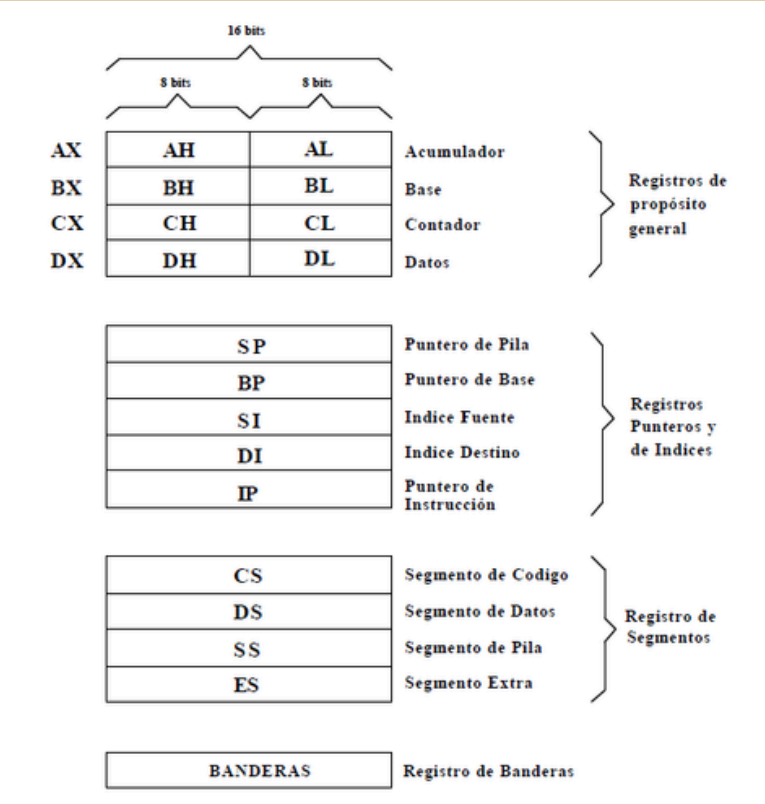


Entonces el generador de código agarra el código intermedio y básicamente empieza a traducirlo en crudo a Lenguaje Máquina

Optimizador de Código

Agarra el ensamblador y mejora el código generado.

```
.file "hola.c"
.section .rodata
.LC0:
.string "Hola elbinario"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $.LC0, %edi
call puts
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (SUSE Linux) 4.8.5"
.section .note.GNU-stack,"",@progbits
```

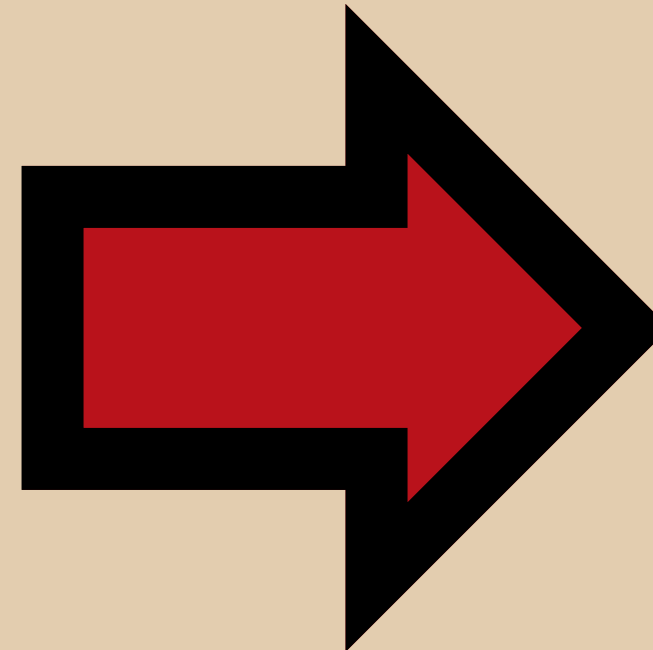


Utiliza instrucciones equivalente pero que llegan a ser más rápidas.
Y utiliza modos de direccionamiento apropiados
Elimina código redundante y también innecesario
Optimiza en tiempo o también puede optimizar en espacio

Optimizador de Código

Somos monstruos
peludos que sabemos
que nos podemos
ahorrar recursos y
tiempo, que tenemos
más registros, podemos
hacer mejores
direccionamientos, y
ahorrarnos
instrucciones

```
; t1 = a + b
MOV AX, [a]
ADD AX, [b]
MOV [t1], AX
; t2 = c - d
MOV AX, [c]
SUB AX, [d]
MOV [t2], AX
; t3 = t1 * t2
MOV AX, [t1]
MUL [t2]
MOV [t3], AX
; x = t3
MOV AX, [t3]
MOV [x], AX
```



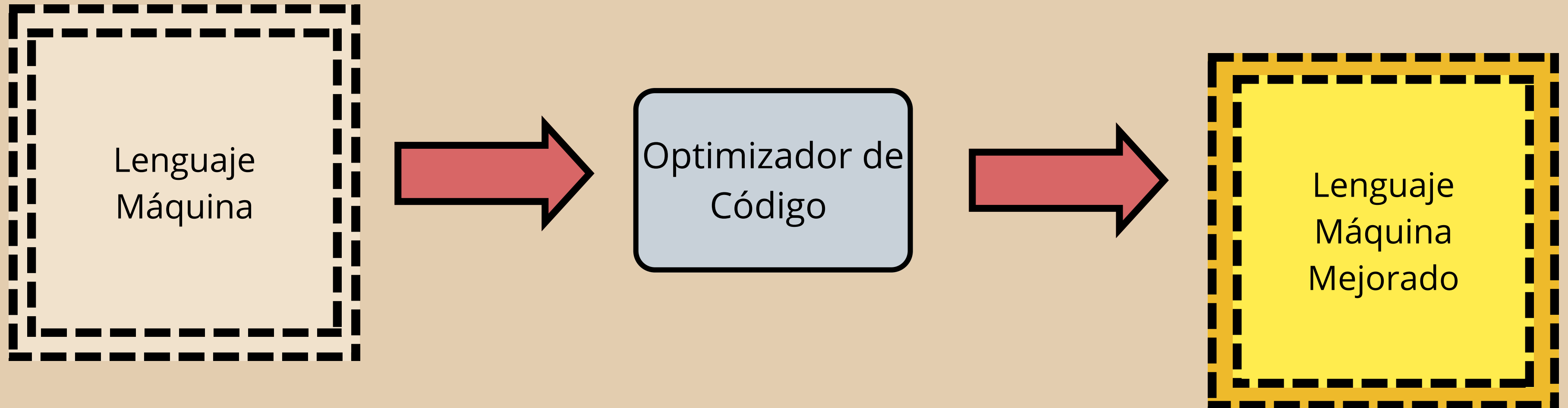
```
MOV AX, [a]
ADD AX, [b]
MOV DX, AX

MOV AX, [c]
SUB AX, [d]

MUL DX

MOV [x], AX
```

Optimizador de Código

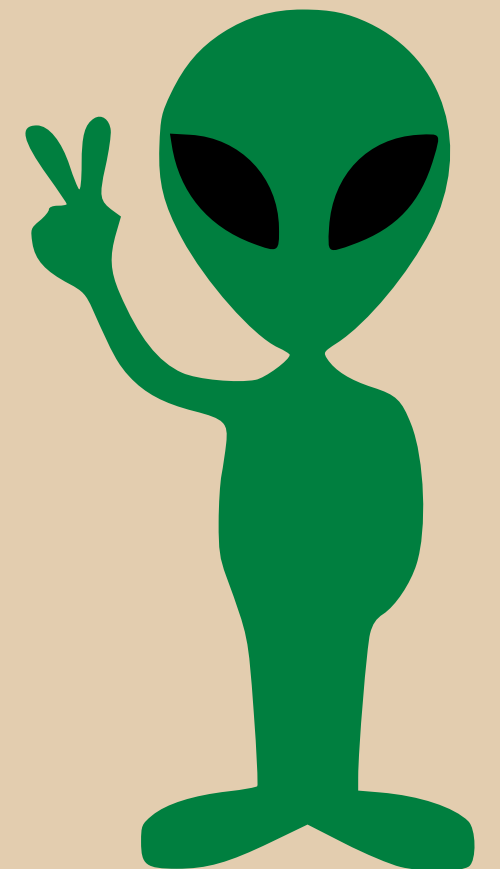


Entonces el optimizador de código agarra el Lenguaje máquina feo y mal hecho por el generador y lo pule y le da amor hasta que lo convierte a Lenguaje Máquina Premium (totalmente mejorado)



**Aquí se acaban las mentiras.
Y podemos dejar de mentir
como Pinocho.**

Los compiladores funcionan
parecido pero no exactamente



iii Muchas gracias!!!

Ian Sebastian Ugalde Muñoz - 2024124076